GREENLEAF CEA · DATA LAYER

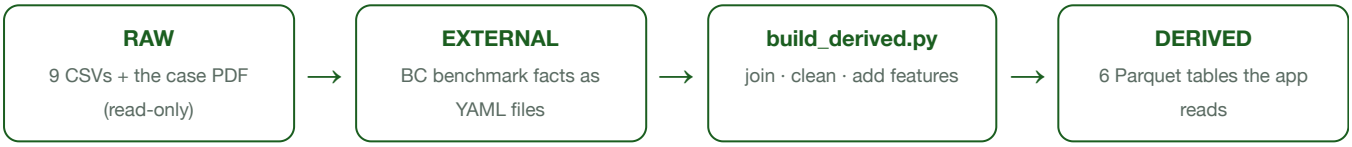
# How the GreenLeaf Data Layer Works

A plain-language tour of the files, the pipeline, and what you can do with it

This is the data foundation for a dashboard built for the RBC × BCCAI × SFU Beedie Agribusiness Analytics Hackathon. The fictional client, **GreenLeaf CEA**, is a British Columbia greenhouse operator running an on-farm experiment: **8 farms**, **20 greenhouses**, and **120 experimental microplots** growing tomato, pepper, cucumber, and strawberry. The question we answer: *does precision agriculture (sensors + alerts + fast responses) actually pay off — and where does it not?*

## 1 · The big idea

Raw data is messy and spread across ten files. Rather than let every chart re-clean and re-join that data (slow, and a recipe for inconsistent numbers), we run **one script** — `build_derived.py` — that does all the cleaning and joining *once* and saves the results as six tidy tables. Every part of the dashboard then reads those six clean tables. One source of truth, no duplicated logic.




## 2 · The folder layout

| Folder / file | What lives there |
|---|---|
| `data/raw/` | The 9 original GreenLeaf CSVs + the case PDF. **Never edited** — treated as read-only truth. |
| `data/external/` | Outside-world BC benchmarks, one small `.yaml` file per fact (e.g. typical greenhouse tomato yield, BC Hydro power rates). |
| `data/derived/` | The 6 clean Parquet tables the script produces. The app only ever reads from here. |
| `build_derived.py` | The one script that turns raw + external into derived. |
| `app/` , `app/tabs/` | The Streamlit dashboard (one module per tab) — built on top of this data layer. |
| `analysis/` , `models/` | Exploration scripts and the trained prediction model. |
| `docs/` , `tests/` | Supporting docs and automated tests that guard the data layer. |

## 3 · What it's made of — the raw ingredients

Nine CSV tables describe the operation at three "zoom levels": the whole farm, each greenhouse, and each individual plot — plus what happened day by day.

| Raw file | Rows | What it holds |
|---|---|---|
| `farms.csv` | 8 | Farm name, region, climate zone, production system. |
| `greenhouses.csv` | 20 | Structure, heating, irrigation, sensor density per greenhouse. |
| `plots.csv` | 120 | The experiment: each plot's crop + assigned treatment. |
| `daily_sensor_readings.csv` | 25,200 | One row per plot per day: temperature, humidity, VPD, stress index, alerts, whether the team acted, how fast. |

| | | |
|---|---|---|
| `daily_input_costs.csv` | 25,200 | Daily cost per plot, split into routine vs. precision spending. |
| `input_applications.csv` | 4,613 | Every individual action taken (fertilizer, water, labor…) flagged routine or precision. |
| `scouting_observations.csv` | 1,800 | Human inspections: blossom counts, pest/disease severity, leaf color. |
| `season_summary.csv` | 120 | End-of-season scorecard per plot: yield, revenue, cost, profit, ROI. |
| `market_and_input_prices.csv` | 840 | External price context for fertilizer, energy, water, etc. |

### The experiment, in one line

Each of the 120 plots was assigned exactly one of **seven treatments** — Control, Low N, High N, Integrated Pest, Reduced Pest, High Light, Shade — and tracked daily for a full season ( `2025-02-15 → 2025-09-12` , 210 days per plot). "Control" is the baseline we measure every other strategy against.

## 4 · How `build_derived.py` works

The script runs four moves, in order:

| Step | What happens |
|---|---|
| **1. Load** | Read all CSVs from `data/raw/` and all `.yaml` files from `data/external/` . Dates are parsed into real date objects (never left as text). |
| **2. Join** | Stitch tables together using shared IDs ( `plot_id` , `greenhouse_id` , `farm_id` ). We only use **LEFT joins** — so if something is missing it shows up as a blank, instead of silently disappearing. |
| **3. Feature-engineer** | On the daily table, sort by plot then date and compute new "smart" columns (explained in §6) — past trends, near-future outcomes, and a prediction target. |
| **4. Write** | Save six Parquet files to `data/derived/` . Re-running overwrites them with identical output (**idempotent** — no random timestamps), so numbers never drift. |

It prints a line per file it writes and a final `[derived] done` . If anything breaks it prints `[derived] FAILED: <reason>` and stops cleanly. Automated tests check the row counts and feature columns every time.

## 5 · The six derived tables

| Derived file | Grain (one row =) | Rows × Cols | Built from |
|---|---|---|---|
| `plot_meta.parquet` | one plot | 120 × 20 | plots + greenhouses + farms — all the "who/what/where" metadata in one place. |
| `season.parquet` | one plot | 120 × 27 | the end-of-season scorecard joined to that metadata. |
| `daily.parquet` | one plot per day | 25,200 × 60 | sensors + daily costs + metadata + the engineered features. The heart of the dashboard. |
| `actions.parquet` | one action | 4,613 × 28 | every input application, labelled routine vs. precision, with plot metadata. |
| `scouting.parquet` | one inspection | 1,800 × 26 | human scouting notes joined to plot metadata. |
| `bc_benchmarks.parquet` | one benchmark | 1+ × 10 | the external YAML facts, each with its source URL and date so claims are traceable. |

## 6 · The "smart" columns in the daily table

These are the columns the raw sensors don't give you — we compute them. They're what let the dashboard look backward, look forward, and predict.

### Looking backward — lags & trends

- `stress_lag_1 / lag_3 / lag_7` — the plant's stress level 1, 3, and 7 days ago. Lets us see momentum.
- `vpd_kpa_7d_mean`, `substrate_moisture_7d_mean`, `pest_pressure_7d_mean`, `canopy_air_diff_7d_mean` — 7-day rolling averages that smooth out daily noise.
- `canopy_air_diff` — how much hotter the leaves are than the air (a classic stress tell).

### Looking forward — the prediction target

- `stress_lead_3` — the stress level 3 days *into the future*.
- `stress_delta_3d_forward` — how much stress is about to change (future minus today).
- `spike_3d` — our headline target: **1 if stress is about to jump by ≥ 0.10 over the next 3 days, else 0.** This is what the predictor model learns to forecast — catching trouble *before* it happens.

**Reality check:** about **18%** of plot-days are real "spikes" (base rate 0.1812) — so the data is meaningfully imbalanced, which the model has to respect.

Because you can't see 7 days into the past on day 1, or 3 days into the future near the end, the first 7 days of each plot have blank lag values and the last 3 days have blank future values — by design, not a bug.

## 7 · How it can be used

The clean data layer powers a four-tab dashboard, each tab aimed at a different audience:

| Tab | For whom | What it does |
|---|---|---|
| **Story** | Farmers | The headline findings — does faster response pay off? what's the dollar value of precision? which plots to watch this week? — with real BC benchmarks overlaid. |
| **Predictor** | Curious / technical | The `spike_3d` model: feed it conditions and it forecasts a stress spike, shows which factors drove the call, and lets you play "what-if". |
| **Under the Hood** | Judges / analysts | The methodology: baselines, model performance, sensitivity checks, and every external benchmark source. |
| **Ask GreenLeaf** | Everyone | A chat assistant grounded in the findings and curated farming docs — ask questions in plain English. |

### In short

Raw greenhouse data → one clean, tested, reproducible data layer → a dashboard that turns sensors, alerts, and costs into a clear answer for a farmer or a lender: **where precision agriculture is worth the money, and where it isn't.**

Generated for GreenLeaf CEA · RBC × BCCAI × SFU Beedie Agribusiness Analytics Hackathon · Data window 2025-02-15 → 2025-09-12 · Figures verified against the live derived build.